

به نام خدا

گزارش کار آزمایش 8 – درس آزمایشگاه طراحی سیستم های دیجیتال

امیر محمد شربتی (402106112) – پوریا رحمانی (402111418)



در ابتدا در آزمایشگاه قرار شد کدی که پیاده سازی کردیم را تست کنیم.

مشکلی که داشتیم این بود که ما هیچ راه ارتباطی با حافظه نداشتیم. در واقع در ماژول تست با به صورت hard code دستورات و دیتای لازم را وارد میکردیم.

پس کد را بدین شکل تغییر دادیم که وقتی مدار ریست میشود، این دستورات و دیتا (باز هم به صورت hard code) در ماژول حافظه در حافظه ریخته شوند. یعنی همین بخش را به بخش ریست حافظه انتقال دادیم:

```
// Operand A = 3 + 2i
DUT.MEM.mem[16] = {16'd3, 16'd2};
ou, yesterday * Uncommitted changes
// Operand B = 1 + 4i
DUT.MEM.mem[17] = {16'd1, 16'd4};

// Instruction: ADD: mem[16] + mem[17] -> mem[3]
DUT.MEM.mem[0] = {4'b0000, 5'd16, 5'd17, 5'd3, 13'd0};

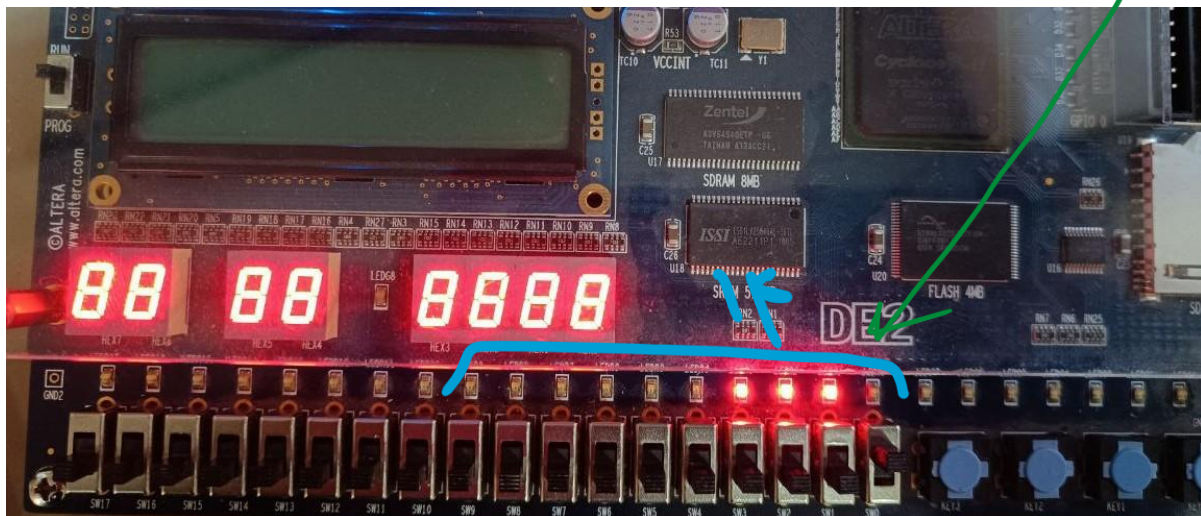
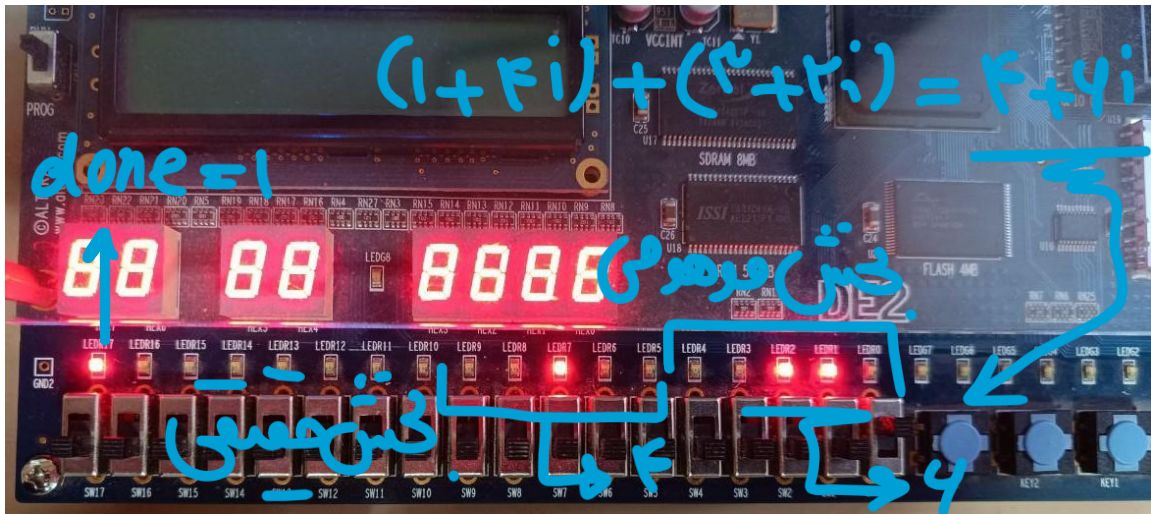
// Instruction: MUL: mem[16] * mem[17] -> mem[4]
DUT.MEM.mem[1] = {4'b0010, 5'd16, 5'd17, 5'd4, 13'd0};

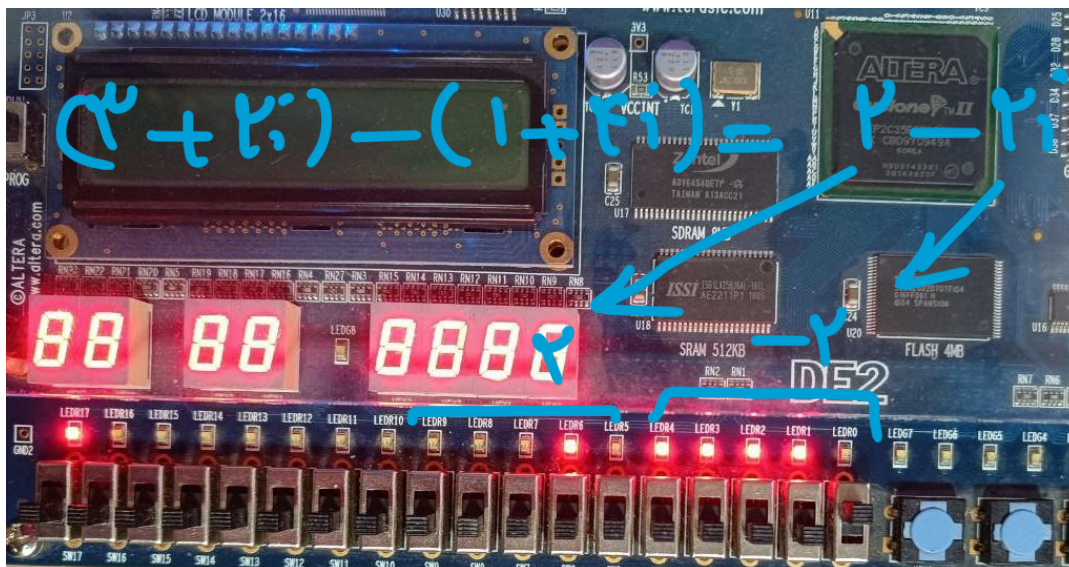
// Instruction: SUB: mem[16] - mem[17] -> mem[6]
DUT.MEM.mem[2] = {4'b0001, 5'd16, 5'd17, 5'd6, 13'd0};
```

مشکل دیگر این بود که ما حاصل را در حافظه ذخیره می کردیم ولی خب باید این را روی برد نشان می دادیم. بنابراین چیزی که قرار بود در حافظه بنویسیم را به خروجی مدار میدادیم تا نتیجه را روی برد ببینیم. یعنی این خط را به مدار اصلی اضافه کردیم: output reg [31:0] mem_wdata;

در مرحله بعد ورودی و خروجی برد را تنظیم کردیم. یعنی key2 به عنوان کلاک، sw0 به عنوان ریست و LEDR17 به عنوان done. خروجی را بدین صورت مشخص کردیم که 5 بیت LEDR0...4 به عنوان بخش موهومی (یعنی متناظر با mem_wdata[0]...[4]) و 5 بیت بعدی، یعنی LEDR5...9 به عنوان بخش حقیقی عدد حاصل (یعنی متناظر با mem_wdata[16]...[20]) هستند.

همانطور که از دستورات مشخص است و در پیش گزارش هم برای ماژول تست توضیح داده شد، دو عدد $1+4i$ و $3+2i$ را در دستورات با هم جمع، تفریق و در هم ضرب میکنیم. این هم نتایج اجرای مدار است که مطابق انتظار است (توجه شود که چون ضرب دو عدد 32 بیتی، 64 بیت است، پس دو خروجی داریم: یکی برای بخش حقیق و دیگری برای بخش موهومی.).





در گام بعد قرار شد که دستور load را اضافه کنیم. بدین معنا که یک عدد در ورودی به مدار بدهیم و این عدد در حافظه ذخیره شود و سپس در محاسبات بعدی مورد استفاده قرار بگیرد. همانطور که فرمودید آدرس را درون برنامه hard code کردیم و فقط از عددی که در ورودی میدهیم استفاده میکنیم.

در مورد پیاده سازی load هم مطابق سایر دستورات 4 بیت سمت چپ را به عنوان opcode در نظر گرفتیم. متناظر با load را 0011 در نظر گرفتیم. سپس 5 بیت مربوط به آدرسی است که در آن عدد را ذخیره میکنیم و مابقی عددی است که در ورودی از کاربر گرفته ایم. بنابراین این state را اضافه کردیم:

```
LOAD: begin
    mem_wr <= 1'b1;
    mem_addr_1 <= instruction[1][27:23];
    mem_wdata <= number;
    dst_addr_4 <= dst_addr_3;
    state <= DONE;
```

توجه شود که در ورودی یک عدد 8 بیتی میگیریم (برای راحتی تست عملی) ولی این را به تعداد بیت های لازم extend میکنیم:

```
wire [31:0] number = {{12{in_number[7]}}, {in_number[7:4]}, {12{in_number[3]}}, {in_number[3:0]}};
```

در گام بعد این مدار را تست کردیم. در واقع یک عدد 8 بیتی در ورودی میدهیم که فرض کردیم 4 بیت سمت راست قسمت موهومی و 4 بیت سمت چپ قسمت حقیقی هستند. سپس در آدرس هارد کد شده در برنامه این عدد را ذخیره میکنیم و سپس دستورات را بر اساس این اجرا میکنیم. توجه شود دستوراتی که در حافظه مقدار دهی میکنیم هم تغییر کردند:

```
// Instruction LOAD: mem[16] = input
mem[0] = {4'b0011, 5'd16, 23'd0};

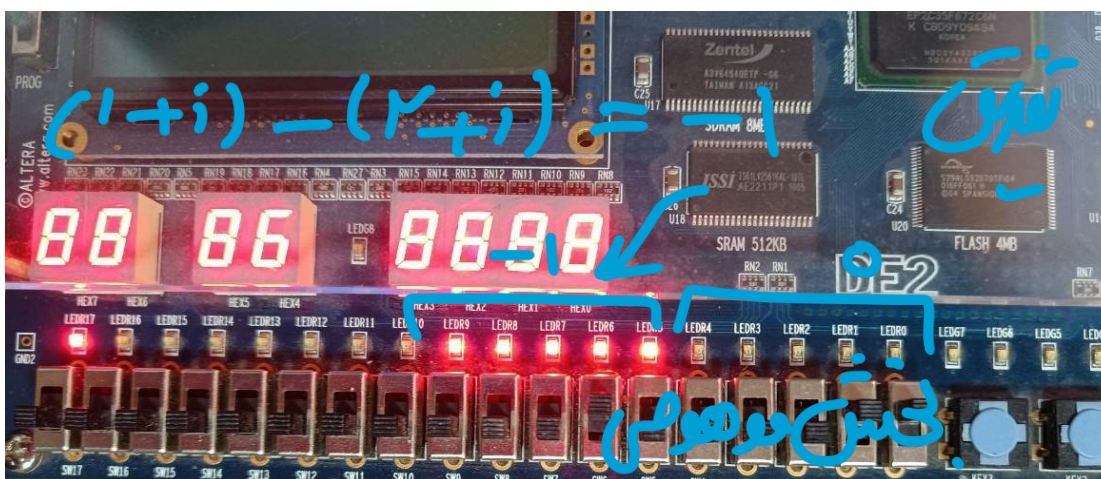
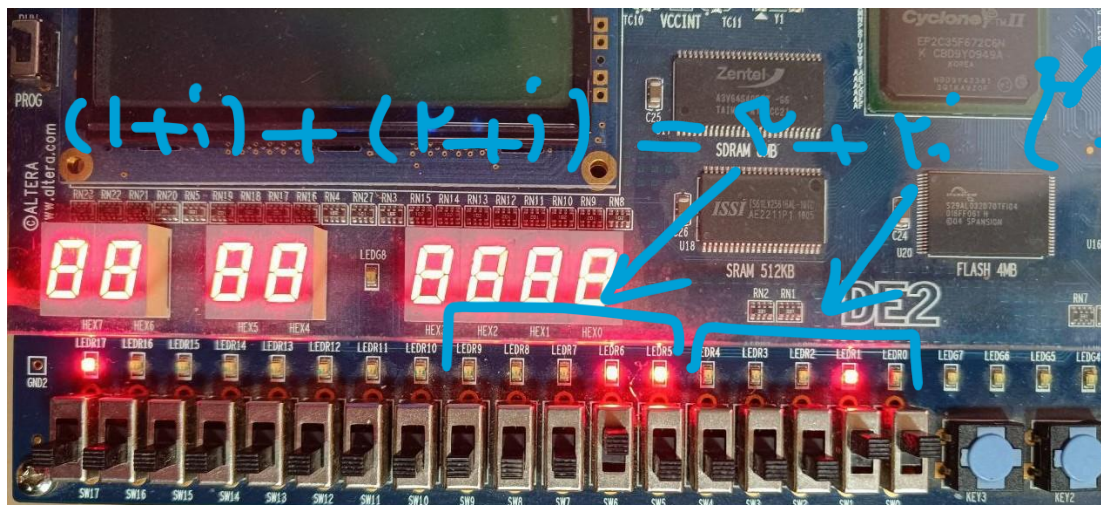
// Instruction LOAD: mem[17] = input
mem[1] = {4'b0011, 5'd17, 23'd0};

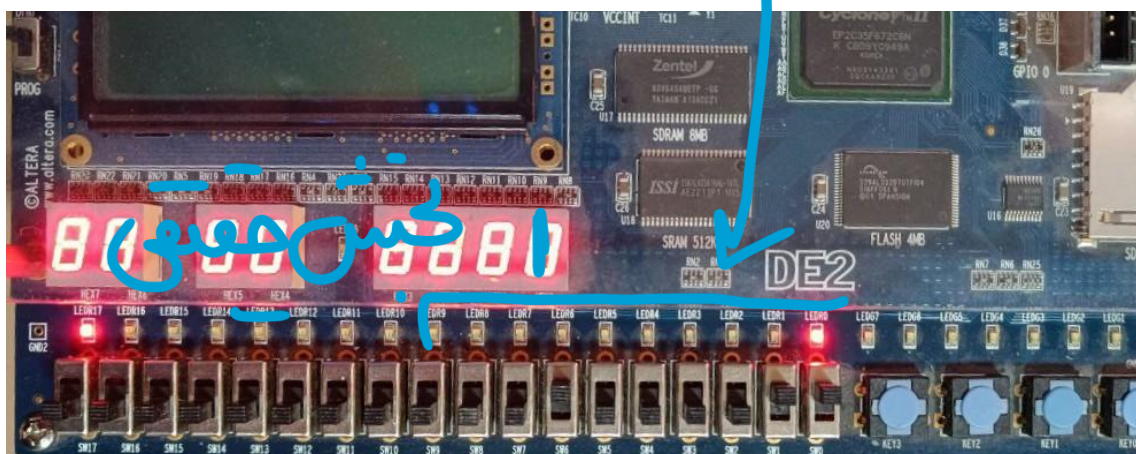
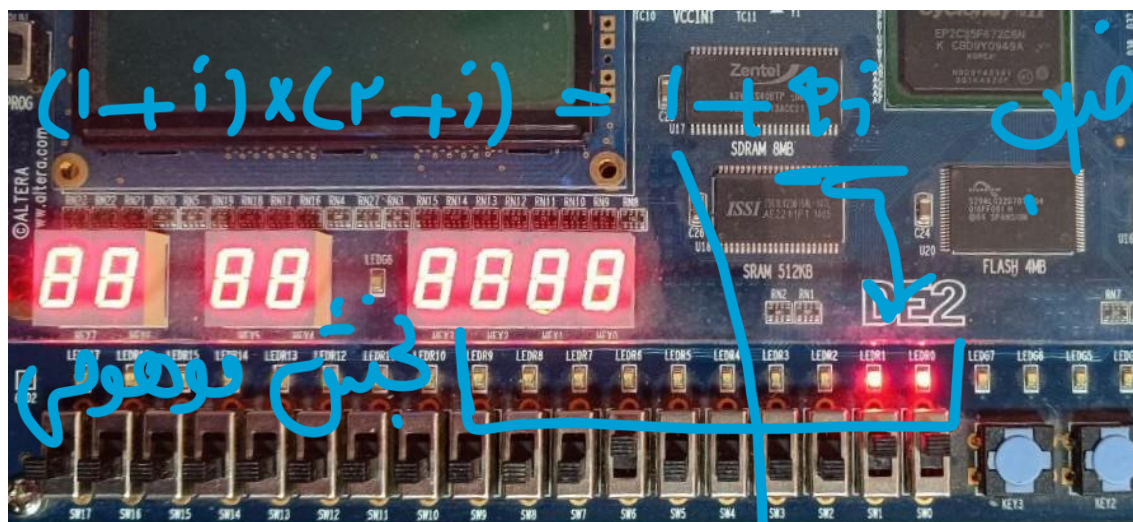
// Instruction: ADD: mem[16] + mem[17] -> mem[10]
mem[2] = {4'b0000, 5'd16, 5'd17, 5'd10, 13'd0};

// Instruction: MUL: mem[16] * mem[17] -> mem[11]
mem[3] = {4'b0010, 5'd16, 5'd17, 5'd11, 13'd0};

// Instruction: SUB: mem[16] - mem[17] -> mem[13]
mem[4] = {4'b0001, 5'd16, 5'd17, 5'd13, 13'd0};
```

اعدادی که در ورودی دادیم اعداد $1+i$ و $2+i$ بودند. خدا رو شکر نتایج درست حاصل شدند:





در نهایت همه نتایج درست است. تنها مشکلی که در کد بود این بود که موفق نشدیم به شیوه ای درست pipeline را پیاده سازی کنیم. ولی خب تلاش خود را انجام دادیم. کد را تا حد خوبی تغییر دادیم و به شیوه pipeline نزدیک کردیم ولی کامل موفق نشدیم مدار را تست کنیم. کد نهایی که در آزمایشگاه به آن رسیدیم را هم به پیوست ارسال کردیم.